

Bevezetés a Rust programozásba

Gembela Gergely

3. óra: A típusrendszer

<https://github.com/bmeaut/VIAUBXAV090>

Primitív típusok

- Integer típusok:
 - Unsigned: `u8`, `u16`, `u32`, `u64`, `u128`, `usize`
 - Signed: `i8`, `i16`, `i32`, `i64`, `i128`, `isize`
- Float típusok (IEEE-754):
 - `f32` és `f64`
- Bool (`bool`)
- Karakter (`char`) **4 byte, unicode kódot reprezentál**

Integer literálok

- Egész számok literáljai:
 - Decimális: `42_42`
 - Hex: `0x2A`
 - Oktális: `0o52`
 - Bináris: `0b0010_1010`
 - Byte: `b'a'` (ASCII)
 - E-notation: `1e4`

Stringek

- Vannak alternatíváink, de alapvetően minden UTF-8
- `&str` (string slice) - a `const char*` megfelelője, de nem azonos
 - 2 eleme van: egy pointer a byte-ok kezdetére, és egy hossz
- `String` - egy heap-allokált tároló
 - növelhető, nagyon sok műveletet támogat
 - slice képezhető egy részére: `let slc = &mystring[1..4];`

String formázás

- `format!()` / `write!()` syntax extension
- Szándékosan hasonló a C `(f|s|...)printf` formázási képességeihez
- Makrók segítségével több helyzetben is egyszerű a stringek formázása
- A támogatott szintaxis [itt](#) található
- Formázáshoz formátumonként saját trait felelős

A (Never) típus

- Viszonylag új stable feature
- Egy kiemelt, értékkel nem rendelkező (nyelvi szinten def.) típus
- Akkor használjuk, amikor egy számításnak nem lesz eredménye
- Típusinferencia esetén hasznos, jellemzően `break` vagy `return` statementek használatakor
- A saját traitek írásakor érdemes lehet a never type számára is implementálni azokat

Enum-ok

- Sokkal erősebb konstrukció mint pl. C/C++-ban
- Egy enummal lényegében azt adjuk meg, hogy az értékünk egyféle lesz a lehetőségek közül
 - Teljesen más adatokat tárolhat minden variáns
 - Mintaillesztéssel ezek után elérhetjük az adott variáns értékét
- Nézzünk erre példákat az előadáshoz tartozó modulban

A tuple-k

- Véges elemű érték, n -es

```
let t1 = (1, 5, "asdf");
```

- Az elemekhez az indexükkel férhetünk hozzá

```
let asdf = t1.2;
```

- Elnevezhetők (pl. `struct Pair(i32, i32)`)
 - de így már nem egyszerű tuple, hanem tuple struct

A slice típus

- Primitív típusnak számít
- Nézet egy folytonos sorozatra
 - Egy pointer és egy hossz alkotja

```
let v1 = vec![42, 43, 44];  
let s1 = &v1[1..];  
let s2 = &mut v1[..]; //nem fordul le, mivel v1 nem mutable, és már van is egy referencia
```

Struktúrák

- A C-hez hasonlóan, de modernebb szintaxissal
 - **Nem C memory layout**
- CamelCase névvel
- A fieldek alapvetően privátok, a `pub` módosítóval publikussá tehetők

```
struct DemoStruct{  
    x: String,  
    y: usize  
}
```

Struktúrák

- A modern architektúrák hatékony használatához (bizonyos esetekben általános használatához) szükség van alignment-re
- A fordító átrendezheti a fieldeket, hogy optimálisabb legyen a padding
- Ezt kézzel megakadályozhatjuk, vagy választhatunk C reprezentációt
 - Ezzel potenciálisan teljesítményt áldozunk fel
- Más nyelvű könyvtárak használata esetén ne felejtsük el!

Struktúrák elrendezése a memóriában

- Jellemzően az alábbiakkal befolyásolható:
 - `#[repr(packed)]`: nincs padding (lehet hogy nem hatékony)
 - `#[repr(C)]`: kényszerítjük a fordítót a C-ben használt layout-ra
 - `#[repr(align(n))]`: kézzel adjuk meg az alignmentet

```
struct A {  
    a: u8,  
    b: u32,  
    c: u16,  
}
```

Trait-ek

- Közös viselkedést írnak le
 - Lehetséges default implementációt adni
 - Hasonló a .NET interface-ekhez
 - Tetszőleges típus számára implementálható
 - Polimorfizmust tesz lehetővé
- Csak akkor implementálható egy típushoz, ha a típus, vagy a trait a lokális crate-ben van definiálva (orphan rule)

Trait implementálása

- Traiteket legegyszerűbben kézzel implementálhatunk

```
impl TraitName for TypeName{  
    //actual impl...  
}
```

- Vagy a fordító is implementálhatja nekünk
 - Ha ezt mégsem szeretnénk:

```
impl !Trait for ReceiverType;
```

Auto traits

- *Sized*: ha egy típus mérete ismert fordításkor
- *Send*: ha egy típus hordozható szálak közt
- *Sync*: ha egy típus referenciái megoszthatók szálak közt
- *Unpin*: ha egy típus nem igényel `pin` garanciát (a `Pin<T>` típussal együtt későbbi előadás anyaga)
- *UnwindSafe* és *RefUnwindSafe*: az unwind safety a hibakezelésnél kerül majd tárgyalásra

A Sized trait

- A legtöbb típus mérete ismert fordításkor
 - Ha nem, dinamikus méretű típus (DST), pl. str, slice-ok, stb.
 - A **Sized** trait-et a fordító automatikusan implementálja
- A változók, konstansok, függvények paramétereit/visszatérési értéke, és statikus értékek csak **Sized** típusúak lehetnek

Derive

- Megkérhetjük a fordítót, hogy implementáljon nekünk traiteket
 - Copy (nem keverendő a Clone-nal, ez csak bitwise copy)
 - Clone
 - Debug
 - Eq
 - Serialize
 - stb.

Inherent methods

- Olyan metódusok, amelyek konkrét típushoz kötöttek (a saját specifikus `impl` blokkjában kerülnek definiálásra)
 - Mindig scope-ban vannak
 - Az első paraméterük jellemzően explicit `&self` vagy `&mut self`

```
impl TAkinekImpl{  
    fn inherent_m1(&self){  
        ...  
    }  
}
```

Interior mutability

- Egy minta, amellyel elérhető, hogy egy típus belső állapota megváltoztatható legyen megosztott referencián keresztül
- Alaphelyzetben követelmény, hogy egy megosztott referencia értéke immutable
- `UnsafeCell<T>` használatával ez a követelmény kikapcsolható, de
 - Vagy futásidejű borrow check-ek biztosítják a biztonságot
 - Vagy unsafe kódhoz vezet

Példa: az `std::sync::atomic` típusai

- A modul *atomic* típusokat tartalmaz
 - Szálközi kommunikációhoz/memóriamegosztáshoz tartalmaz primitíveket
 - Az írásuk és olvasásuk a C++-ból már potenciálisan ismert **Ordering** megadását igényli, de **ez nem az interior mutability következménye**
 - A fordító, és egyes esetekben a CPU is átrendezhet instrukciókat, erre, és különböző területek szinkronizációjára van hatással

Castolás és konverzió

- Nincs implicit típuskonverzió (pl. `u64` -> `i32`), de kézzel az `as` operátorral castolhatunk
- A castolás alapvetően C szabályok szerint megy, kivéve az undefined behavior eseteket
- Konverzió a `From` és `Into` traiteken keresztül lehetséges

Union

- Rust-ban is létezik, a C-hez hasonló működéssel
- Pattern matching is használható az értékek olvasására, de ez is unsafe, mivel nem tudjuk melyik mező aktív
- Csak unsafe módban olvashatók a fieldek
- Minden érték azonos lifetime-mal rendelkezik, ezért nem lehet fieldenként külön referenciánk
- Vannak megkötések az elemek típusaira

Aliasing

- A `type <new_name> = <name>;` statement segítségével alternatív neveket adhatunk a típusoknak
 - Az átnevezett típusok is CamelCase nevet kell kapjanak
 - Praktikus típusparaméterek egyszerűsítésére
 - pl. az `anyhow::Result<T>` a `Result<T, anyhow::Error>` megfelelője
- Lehetséges újraexportálni

Típusinferencia

- Sok esetben elhagyhatjuk a típusok explicit megadását, mert a fordító képes levezetni azokat
 - Komplex, több kifejezés közt is fenntartott kontextussal oldja fel a várt típusokat
 - Ha nem értünk vele egyet, érdemes explicit megadni a várt típust, ezzel garantáljuk hogy a megfelelő helyen, és jellemzően részletesebb hibát/indoklást kapunk
- Az *életciklusok* tárgyalásakor visszatérünk a fontosságára

W3T1: Ismerkedjünk meg a `serde`-vel

- A Serde egy keretrendszer, ami általánosan és hatékonyan tud **serializálni** és **deserializálni** struktúrákat
- Rengeteg célformátum van, a legismertebbek a JSON, YAML, TOML, és a BSON, de több bináris formátumot is támogat (pl. Postcard)
- Készítsünk egy új projektet, majd hozzunk létre legalább 2 saját struktúrát, majd a `#derive` attribútum segítségével implementáljuk a `serde Serialize` és `Deserialize` trait-jeit
- A feladat további követelményei a GitHub oldalon (lesznek) elérhetők

W3T2: Az egyik legnépszerűbb Prog 1 házi

- Az óra hátralévő részében készíts egyszerű raktárkezelőt
 - Használhatod a `clap` könyvtárat CLI argumentumok feldolgozására
 - Használj saját struktúrákat az adatmodellhez
 - Tárold a bejegyzéseket structokat tároló vectorokban, a Serde segítségével serializáld, majd mentsd fájlba az adatokat

Köszönöm a figyelmet!